

Día D — qué ejecuta cada uno

Ramas y fases ya están repartidas. Cada quien corre **un solo comando de agente**: `/speckit-implement` con su fase. Lo único que se escribe a mano es **la Historia que dicte el profesor**.

Quién	Su rama	Su fase	Archivos que le pertenecen — nadie más los toca
Sergio	main	reparte	spec.md, tasks.md
Mateo	feature/hu-ui	fase 6	app/*/page.jsx, components/**, hooks/**
Johan	feature/hu-api	fase 7	app/api/**/route.js y sus .test.js
Tomás	feature/hu-datos	fase 8	test/**, lib/**
Santiago	main	merge y deploy	scripts/**

Paso 0 · los cinco

20 minutos antes de clase

```
cd ~/Documents/web/speckit-ai-generator
git checkout main
git pull origin main
npm install
npm test # 0 failed
npx -y react-doctor@latest . --score # 100
```

La última línea baja el paquete: si no la corres ahora, en vivo se tarda un minuto largo. Y ese **100** es el número que hay que devolver al final.

Santiago, además — deja el panel de Vercel abierto y la URL cargada en el celular:

```
npm run smoke https://ia-generator-openspec.vercel.app
```

Y cada quien deja abierto `specs/001-ai-media-generator/tasks.md` en la fase que le toca: es lo que se señala cuando pregunten de dónde salió el código.

Paso 1 · Sergio mete la Historia y reparte las fases

≈ 2 min · rama **main**

```
git checkout main
git pull origin main
```

1.1 — la Historia entra al spec. Pega esto con la historia (o las dos) literales del profesor:

```
/speckit-specify Agrega al final de la seccion User Scenarios de
specs/001-ai-media-generator/spec.md las User Stories que siguen, numeradas
desde User Story 5.
HU 5: "historia literal que dictó el profesor"
# si dictó una segunda, agrega esta línea; si no, bórrala
HU 6: "segunda historia literal"
Cada una con sus Acceptance Scenarios numerados en Given/When/Then, y agrega
los Functional Requirements que hagan falta.
NO crees una feature nueva. NO corras create-new-feature.sh. NO cambies de rama.
NO escribas ni modifiques .specify/feature.json.
NO toques plan.md ni tasks.md. Solo edita ese spec.md que ya existe.
```

Verifica esto antes de seguir. `/speckit-specify` por dentro dice que *siempre* crea una carpeta nueva y reescribe `.specify/feature.json`. Si lo hace, los tres `/speckit-implement` apuntan a un `tasks.md` vacío y no construyen nada.

```
cat .specify/feature.json # tiene que decir specs/001-ai-media-generator
git status --short # solo spec.md modificado, ninguna carpeta specs/002-...
```

Si salió una carpeta nueva: `rm -rf specs/002-*` y `git checkout .specify/feature.json`, y sigue.

1.2 — el reparto. Este prompt se pega tal cual, sin cambiar nada:

Agrega al final de `specs/001-ai-media-generator/tasks.md` cuatro fases nuevas para las User Stories que acabamos de agregar. No regeneres el archivo ni toques las fases 1 a 5. Numera las tareas desde T040.

```
## Fase 6 – pantallas    Owns: app/*/page.jsx, components/**, hooks/**
## Fase 7 – API          Owns: app/api/**/route.js y sus .test.js
## Fase 8 – datos        Owns: test/**, lib/**
## Fase 9 – merge        Owns: scripts/**
```

Antes de las tareas, escribe el contrato HTTP entre la Fase 6 y la Fase 7: ruta, metodo, forma exacta del cuerpo de respuesta y código para el anónimo. Ese contrato no se negocia después.

Reglas: ningún archivo puede aparecer en dos fases. Cada tarea dice que Acceptance Scenario cubre, con la etiqueta [US5-2], y es test-first: primero la prueba, nombrada `it("US5-2: ...")`, después el código. Ninguna fase puede importar un símbolo nuevo de un archivo que le pertenece a otra fase — si lo necesita, lo define en su propio archivo. Si una HU no da trabajo para alguna fase, deja esa fase con una sola tarea que lo diga.

```
git add specs/001-ai-media-generator/
git commit -m "HU en vivo: User Stories nuevas y sus fases 6, 7, 8 y 9"
git push origin main
```

Después dice en voz alta: «fase 6 Mateo, fase 7 Johan, fase 8 Tomás — arranquen».

Paso 2 · Mateo, Johan y Tomás

≈ 6 min · cada uno en su máquina

Copia solo tu columna, de arriba abajo. El agente hace el resto:

MATEO

```
git checkout main
git pull origin main
git checkout -b feature/hu-ui

/speckit-implement fase 6

npm test
npx -y react-doctor@latest . --score
git add -A
git commit -m "feat: HU en vivo pantallas"
git push -u origin feature/hu-ui
```

JOHAN

```
git checkout main
git pull origin main
git checkout -b feature/hu-api

/speckit-implement fase 7

npm test
npx -y react-doctor@latest . --score
git add -A
git commit -m "feat: HU en vivo API"
git push -u origin feature/hu-api
```

TOMÁS

```
git checkout main
git pull origin main
git checkout -b feature/hu-datos

/speckit-implement fase 8

npm test
npx -y react-doctor@latest . --score
git add -A
git commit -m "feat: HU en vivo datos"
git push -u origin feature/hu-datos
```

Es `/speckit-implement`, no `/implement`, y siempre con la fase — sin argumento el agente construye todo el backlog. **Rojo en `npm test`, o un score por debajo de 100, es no-push:** dile al agente qué falló, que lo arregle y vuelva a correr los dos. Cuando tu push termine, anúncialo en voz alta.

Lo que se rompió en el ensayo, y lo que hay que decirle al agente si pasa.

- **Pantalla nueva que lee datos al montar** — las pruebas viejas de esa pantalla sustituyen `fetch` por orden, y la lectura nueva se come la primera respuesta encolada. Cinco pruebas que nada tienen que ver se ponen rojas. Se arregla en ese mismo archivo: que el doble responda **por método**, no por turno.
- **Score que baja de 100** — `react-doctor` castiga un `fetch` dentro de un `useEffect` en un `page.jsx`, y un `<div onClick>` haciendo de modal. Sácale el `fetch` a un hook en `hooks/` y usa `<dialog>`. También castiga `.filter().map()` encadenados — un `flatMap`. Con el baseline en 100, una sola advertencia deja el CI rojo y **no despliega**.
- **Nombres accesibles que chocan** — dos imágenes con `alt` parecido, o un botón nuevo cuyo nombre accesible sale de su `alt`, rompen `getByRole` de pruebas viejas. Ponle `aria-label` explícito al botón nuevo.
- **Helper de pruebas que no exporta lo nuevo** — si el archivo de la ruta tiene un `load()` que devuelve `{ POST, ...mocks }`, agregar un `GET` sin agregarlo ahí falla con `GET is not a function` y parece un error de la ruta.
- **Importar algo que otro está escribiendo ahora** — nadie importa un símbolo nuevo de `test/fixtures.js` ni de un archivo de otra fase. Datos de ejemplo, en tu propio archivo.

Solo cuando los tres hayan dicho que subieron. En este orden — datos, API, pantallas:

```
git checkout main
git pull origin main
git fetch origin
git merge --no-ff origin/feature/hu-datos
git merge --no-ff origin/feature/hu-api
git merge --no-ff origin/feature/hu-ui
```

Uno por línea, esperando a que termine cada uno. En el ensayo los tres entraron sin un solo conflicto: ningún archivo aparece en dos listas de *Owns*. Si un merge se enreda: `git merge --abort`, y esa rama salió de su fase.

Con los tres adentro, los cuatro chequeos — el score también:

```
npm run lint
npm test
npx -y react-doctor@latest . --score
npm run build
git push origin main
```

Ese push despliega en Vercel solo. Mientras construye, proyecta el grafo de ramas:

```
git log --graph --oneline -12
```

Paso 4 · todos validan sobre el despliegue

≈ 1 min

1. **Santiago** espera el check verde del deploy en Vercel (1–2 min) y corre:

```
npm run smoke https://ia-generator-openspec.vercel.app
```

2. Todos abren `https://ia-generator-openspec.vercel.app` en el celular, con datos móviles, y recorren la Historia nueva de punta a punta.
3. Si preguntan de dónde salió el código, el dueño abre `tasks.md` en su fase, señala la etiqueta `[US5-2]` y corre esa sola prueba:

```
npm test -- -t "US5-2"
```

4. Si preguntan qué criterio de aceptación demuestra esa prueba, se lee el `Given/When/Then` numerado en `spec.md`, en la User Story que el profesor acaba de dictar. Ese es el camino completo: **necesidad** → **spec** → **tasks** → **prueba** → **código** → **commit** → **desplegado**.